SERVERLESS CONNECTION

Group 4

Presented by,
Bhavadhaarany Ragupathy
Sarita Rani

SERVER CONNECTION

What is server connection?

- A server is a powerful computer that gives us information when we ask for it.
- It waits for your request, then responds with what you asked for.
- This communication is called a server connection.

Example:

- Sends a website when you open a browser.
- Gives you data when you open an app like Instagram.

SERVER CONNECTION

Key Terms:

- Client
- Server
- request
- response
- connection

Types of Server Connection

- HTTP/HTTPS
- Web Socket
- FTP
- SSH

SERVERLESS CONNECTION

What is serverless connection?

- A model where developers focus on code, while the cloud provider manages servers.
- Functions are triggered by events (FaaS).
- Scales automatically with demand.

Example:

- Automatically sends a notification when someone likes your post.
- Resizes a photo you just uploaded, instantly.

SERVERLESS CONNECTION

Benefits of Serverless Processing

- Auto-scaling: Handles more users automatically.
- Pay-per-use: Only pay when your code runs.
- Less maintenance: No server patching or updates.
- Faster to deploy: Quickly release new features.
- Better reliability: Cloud handles failover and backups.

SERVERLESS VS TRADITIONAL SERVER

Feature	Traditional Server	Serverless
Provisioning	Manual	Auto-scaled
Cost	Pay always	Pay per request
Maintenance	Manual updates	Managed by provider
Scaling	Manual or auto	Fully automatic

SERVERLESS CONNECTION

Serverless Platforms Overview

Popular Platforms:

- AWS Lambda
- Azure Functions
- Google Cloud Functions

Emerging Frameworks:

- Hiku
- Dapr
- AARCH
- Hacher

Why so many?

- Different cloud providers and frameworks solve different needs.
- Choose based on your app’s requirements (e.g. event-driven, microservices, data processing).

Technology 1 - HIKU

What is Hiku?

- Hiku is a serverless data query engine that helps you fetch and aggregate data from multiple microservices in a single query.
- It’s similar to GraphQL, but designed to be lightweight and high-performance.
- Supports composable queries, so you can combine data from different sources easily.
- Ideal for building modern web and mobile apps that rely on microservices architecture.

How Hiku Works?

- Hiku is a serverless data query engine that processes hierarchical queries, similar to GraphQL.
- It allows applications to fetch nested data from multiple microservices in a single request.
- This reduces the need for multiple round trips and improves performance.
- Hiku supports plugins to add features like caching, logging, or authentication as needed.
- It’s designed to be lightweight and high-performance, making it suitable for serverless deployments.
- Overall, Hiku simplifies data fetching for modern web and mobile apps that rely on microservices.

Advantages of Hiku

Highly efficient query resolution

- Hiku optimizes how queries are processed and minimizes unnecessary data fetching.

Flexible

- Works with various data sources like REST APIs, databases, and other services.

Reduces over-fetching and under-fetching

- Fetches only the data you need, avoiding too much or too little data in a response.

Composable queries

- Supports modular design by letting you build queries from smaller pieces, making it easier to manage and reuse.

Disadvantages of Hiku

Learning curve for developers new to hierarchical query systems

- Hiku's query composition is different from traditional REST or simpler GraphQL setups, so developers need to spend time understanding how to structure queries effectively.

Requires careful query design

- If queries are not optimized, they might fetch more data than necessary or make multiple backend calls, which can impact performance.

Smaller community compared to mainstream frameworks

- Hiku has a niche user base, meaning fewer tutorials, fewer community plugins, and less community support compared to frameworks like GraphQL or popular REST frameworks.
- This can sometimes make troubleshooting or finding best practices more challenging.

When to Use Hiku?

- Use Hiku when you need to query multiple microservices in a single request.
- It’s perfect for avoiding over-fetching and under-fetching, ensuring you get only the data you need.
- Best suited for aggregating data from different services in microservice-based apps.

TECHNOLOGY 2 - DAPR

What is Dapr?

- Dapr stands for Distributed Application Runtime.
- It’s an open-source, portable, event-driven runtime for building modern microservice-based applications.
- Helps you abstract away common challenges like state management, service discovery, pub/sub messaging, and secret management.
- It’s designed to be language-agnostic and works with any framework, which means developers can use their preferred programming language.
- Ideal for building reliable and scalable distributed apps that need to communicate efficiently and consistently.

How Dapr Works?

Runs as a sidecar alongside each microservice.

- This means Dapr runs in its own container or process next to the app, intercepting requests and adding features without changing app code.

Exposes a set of **building blocks** that handle common distributed system patterns:

- **State Management**: Easily store and retrieve app state in a consistent way.
- **Service Discovery & Invocation**: Find and call other services dynamically.
- **Pub/Sub Messaging:** Publish and subscribe to events easily.
- **Bindings & Secrets:** Connect to external systems and manage secrets securely.
- Because it's **language-agnostic**, developers can build microservices in any language they prefer (e.g., Python, Java, Node.js).
- Dapr **simplifies integration**, so developers can focus on business logic rather than infrastructure concerns.

Advantages of Dapr

- **Simplifies distributed system challenges—**you don’t have to reinvent the wheel for state, messaging, or discovery.
- **Extensible via building blocks,** so you can pick and choose the features you need and even swap implementations (e.g., Redis, Kafka, etc.).
- **Language-agnostic—**works with any programming language, promoting flexibility in team skill sets.
- **Portable—**runs on Kubernetes, Docker, or even bare metal, supporting cloud, on-premises, or hybrid deployments.
- **Accelerates development** by providing standardized ways to solve common problems.

Disadvantages of Dapr

- **Adds complexity t**hrough the sidecar architecture, which means you need to manage additional containers or processes per microservice.
- Can introduce **resource overhead,** especially in resource-constrained environments (each sidecar consumes CPU and memory).
- **Learning curve—**teams transitioning from monolithic to microservices may need time to understand Dapr’s concepts and integrate them effectively.
- Potential **latency** overhead for some features (e.g., communication between app and sidecar).

When to Use Dapr?

- When building **cloud-native, microservices-based applications** that need to be **resilient, scalable, and portable.**
- When your app requires **consistent state management** across distributed services **or event-driven architecture.**
- When you need to simplify **service-to-service communication** and avoid writing custom logic for discovery, retries, and timeouts.
- When you want to **standardize microservice interactions** and integrate with existing cloud services (e.g., Azure, AWS, GCP).

HIKU VS DAPR COMPARISON

Feature	Hiku	Dapr
Focus	Serverless data query engine	Distributed microservices runtime
Architecture	Query composition, GraphQL-like	Sidecar pattern alongside each service
Use Case	Efficient data aggregation from multiple services	Service-to-service communication, state, messaging, and orchestration
Learning Curve	Moderate—focused on queries	Steeper—many building blocks to understand
Deployment	Lightweight serverless function or container	Typically sidecar in Kubernetes or containers

TECHNOLOGY 3 - SLOPE

What is SLOPE?

SLOPE stands for **Source, Load, Organize, Process, Extract.**

- It's a cloud-based platform designed to help teams onboard and manage datasets with traceability and version control.
- Users define assumptions for data (like field meanings, units, or formats), track how data is transformed, and connect output to platforms like Snowflake.
- Great for data teams in insurance, finance, and analytics who need repeatable and auditable pipelines.

How SLOPE Works

- we start by uploading your raw data (Excel, CSV, etc.)
- In SLOPE, we define input assumptions (e.g., what each column means).
- It automatically versions each upload and change.
- we can send the cleaned and tracked data directly to tools like Snowflake for analytics.

ADVANTAGES OF SLOPE

- **Assumption Tracking**
  Define and document data structure, units, and logic.
- **Data Versioning**
  Easily revert, compare, or reprocess past datasets.
- **Built-in Collaboration**
  Teams can review, comment, and approve assumptions together.
- **Integrates with Snowflake**
  Sends processed datasets directly to cloud warehouses.

DISADVANTAGES OF SLOPE

1. Proprietary (Not Open-Source)

2. Limited Customization Without Paying

3.Focused on Batch Processing

- **Not an open source** :What it means: SLOPE is a commercial software product. we can’t access or modify the source code, unlike open-source tools like Apache Airflow
- **Why it matters:** If your team needs very custom features or integrations, you’re limited to what SLOPE offers (unless you pay for enterprise features).
- **Impact:** This could slow down innovation or increase costs if you outgrow the platform's base functionality

2. Limited Customization Without Paying

- What it means: Many useful features like advanced analytics, automation, and third-party integrations (e.g., APIs or direct database syncing) may be locked behind a paywall.
- Why it matters: For startups or academic projects, this limits what you can do unless you subscribe to premium tiers.
- Impact: You may find that while the basic platform is helpful, more complex workflows or automation are only available at higher pricing levels.

Focused on Batch Processing

- What it means: SLOPE is designed to handle bulk data uploads (e.g., Excel files or CSVs) that are processed at once — not continuous or live data.
- Why it matters: If you need to process real-time data (e.g., streaming news feeds, live IoT data), SLOPE won't support that use case.
- Impact: You may need to use other tools for real-time ingestion and reserve SLOPE only for structured, periodic data loads.

When to Use SLOPE?

- When working with regulated or auditable datasets.
- For teams that need to track data transformations and logic over time.
- When collaborating across departments to define data meaning (e.g. data scientists).

TECHNOLOGY 4 - AARCH (ARM64 SERVERLESS)

What is AARCH?

- AARCH stands for ARM64 architecture – a type of processor (CPU) that is more efficient and cheaper than older ones like x86.
- Big cloud providers like AWS now let we run our code (like Lambda functions) on ARM64 processors instead of x86.
- AARCH is great for apps that do a lot of data processing, like running models, transforming data, or handling lots of user requests.

- **It's like switching from a big, expensive gas car (x86) to a smaller, cheaper electric car (ARM64) — it gets the job done faster and for less money.**

How AARCH Works

- Choose arm64 as the architecture when deploying AWS Lambda.
- Code runs on Graviton CPUs—ARM-based processors built for efficiency.
- It works the same as regular Lambda but uses less power and costs less.

Same functionality, just faster and cheaper.

ADVANTAGES OF AARCH

- **Lower Cost**
- ARM64 Lambda functions are up to 30% cheaper to run than x86.
- **Faster for Short Jobs**
- Especially good for ML inference, ETL, or API response handling.
- **Eco-Friendly**
- Graviton processors consume less power.

DISADVANTAGES OF AARCH

1. Compatibility Issues:

2. Needs Testing:

3. Newer Architecture:

Together, these problems can lead to longer development times, higher costs, unstable systems, or worse performance — all of which can seriously impact business or project success

Let me explain these three more...

1. Compatibility Issues:
Some Python, R, or Node packages don’t work well or aren’t ready for ARM64 yet, so we might face problems running your code.

2. Needs Testing:
Moving from x86 to ARM64 means you have to test your code carefully to make sure everything works correctly on the new system.

3.Newer Architecture:

ARM64 is a relatively recent design compared to the older and more established x86 processors. Because it’s newer, the software ecosystem—like developer tools, libraries, debugging options, and community resources—is still growing. This means you might find fewer ready-made solutions, less documentation, or fewer experts who know how to solve problems on ARM64. Over time, this support will improve, but right **now it can make developing, troubleshooting, and optimizing software a bit harder compared to the mature x86 systems.**

When to Use AARCH?

Use AARCH when you want to run a lot of data-processing functions efficiently — it’s cheaper, faster, and works well for real-world projects like scraping news and analyzing sentiment

It’s especially useful when your functions run often or need to handle large datasets, because ARM64 processors are optimized for both performance & energy savings. AARCH is a smart choice for students, startups.

SLOPE vs. AARCH

Feature	Slope	AARCH
Purpose	Data onboarding, assumption tracking, versioning	Efficient compute for serverless functions
Type	Data processing platform (SaaS)	CPU architecture for serverless compute
Real time support	❌ Mostly batch processing	✅ Real-time event handling
Open source	❌ Proprietary(Not an open source)	✅ AWS Lambda supports ARM64 publicly
Use in capstone	Managing cleaned news data and its structure	Running sentiment analysis, transformation jobs

These tools together support different layers of a serverless data pipeline — from data ingestion (SLOPE) to communication (DAPR), query optimization (HIKU), and high-performance compute (AARCH) — giving you the power to build efficient, scalable, and modern cloud applications.

THANK YOU